

# API\_DOCUMENTATION

## HelixCode API Documentation

**Version:** 1.0.0 **Base URL:** `http://localhost:8080/api/v1` **Authentication:** Bearer Token (JWT)

---

### Overview

HelixCode provides a comprehensive REST API for managing distributed AI development workflows. This document covers platform-specific endpoints for Aurora OS (security-focused) and Harmony OS (distributed computing).

---

### Authentication

All API requests require authentication via Bearer token.

#### Login

**Endpoint:** POST `/api/v1/auth/login`

**Request:**

```
{
  "username": "admin",
  "password": "your_password"
}
```

**Response (200 OK):**

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "expires_at": "2025-11-08T10:00:00Z",
  "user": {
    "id": "user-123",
    "username": "admin",
    "roles": ["admin"]
  }
}
```

## Using the Token

Include in all requests:

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

---

## Core Endpoints

### Tasks

#### Create Task

**Endpoint:** POST /api/v1/tasks

**Request:**

```
{
  "title": "Implement user authentication",
  "description": "Add JWT-based auth system",
  "type": "building",
  "priority": "high",
  "project_id": "project-123"
}
```

**Response (201 Created):**

```
{
  "id": "task-456",
  "title": "Implement user authentication",
  "status": "pending",
  "priority": "high",
  "created_at": "2025-11-07T10:00:00Z"
}
```

#### Get Task

**Endpoint:** GET /api/v1/tasks/{id}

**Response (200 OK):**

```
{
  "id": "task-456",
  "title": "Implement user authentication",
  "description": "Add JWT-based auth system",
  "type": "building",
  "priority": "high",
  "project_id": "project-123",
  "status": "pending",
  "created_at": "2025-11-07T10:00:00Z"
}
```

```
"priority": "high",
"status": "running",
"assigned_worker": "worker-789",
"progress": 45,
"created_at": "2025-11-07T10:00:00Z",
"updated_at": "2025-11-07T10:30:00Z"
}
```

## List Tasks

**Endpoint:** GET /api/v1/tasks

**Query Parameters:** - status (optional): Filter by status (pending, running, completed, failed) - priority (optional): Filter by priority (low, normal, high, critical) - limit (optional): Number of results (default: 50) - offset (optional): Pagination offset (default: 0)

**Response (200 OK):**

```
{
  "tasks": [
    {
      "id": "task-456",
      "title": "Implement user authentication",
      "status": "running",
      "priority": "high"
    }
  ],
  "total": 1,
  "limit": 50,
  "offset": 0
}
```

## Update Task

**Endpoint:** PUT /api/v1/tasks/{id}

**Request:**

```
{
  "status": "completed",
  "progress": 100
}
```

**Response (200 OK):**

```
{
  "id": "task-456",
  "status": "completed",
  "progress": 100,
  "updated_at": "2025-11-07T11:00:00Z"
}
```

## Delete Task

**Endpoint:** DELETE /api/v1/tasks/{id}

**Response** (204 No Content)

---

## Workers

### Register Worker

**Endpoint:** POST /api/v1/workers/register

**Request:**

```
{
  "hostname": "worker-node-1",
  "capabilities": ["planning", "building", "testing"],
  "resources": {
    "cpu": 8,
    "memory": 16384,
    "gpu": true
  }
}
```

**Response** (200 OK):

```
{
  "worker_id": "worker-789",
  "registered_at": "2025-11-07T10:00:00Z"
}
```

### Worker Heartbeat

**Endpoint:** POST /api/v1/workers/heartbeat

**Request:**

```
{
  "worker_id": "worker-789",
  "status": "idle",
  "current_task": null,
  "resources": {
    "cpu_usage": 15.5,
    "memory_usage": 4096
  }
}
```

**Response (200 OK):**

```
{
  "acknowledged": true,
  "assigned_task": null
}
```

## List Workers

**Endpoint:** GET /api/v1/workers

**Response (200 OK):**

```
{
  "workers": [
    {
      "id": "worker-789",
      "hostname": "worker-node-1",
      "status": "idle",
      "last_heartbeat": "2025-11-07T10:05:00Z",
      "capabilities": ["planning", "building", "testing"]
    }
  ]
}
```

---

## Aurora OS Specific Endpoints

### Security Audit Logs

#### Get Audit Logs

**Endpoint:** GET /api/v1/aurora/audit

**Query Parameters:** - event\_type (optional): authentication, authorization, data\_access, config\_change - start\_time (optional): ISO 8601 timestamp - end\_time (optional): ISO 8601 timestamp - limit (optional): Number of results (default: 100)

**Response (200 OK):**

```
{
  "logs": [
    {
      "id": "audit-123",
      "event_type": "authentication",
      "user_id": "user-123",
      "action": "login",
      "result": "success",
      "ip_address": "192.168.1.100",
      "timestamp": "2025-11-07T10:00:00Z",
      "metadata": {
        "user_agent": "Mozilla/5.0..."
      }
    }
  ],
  "total": 1
}
```

## Security Metrics

**Endpoint:** GET /api/v1/aurora/metrics/security

**Response (200 OK):**

```
{
  "auth_attempts": {
    "successful": 1250,
    "failed": 15,
    "rate": 0.012
  },
  "active_sessions": 42,
  "encryption_status": "active",
  "security_level": "enhanced",
  "last_audit": "2025-11-07T09:00:00Z"
}
```

## System Monitoring

**Endpoint:** GET /api/v1/aurora/monitoring/system

**Response** (200 OK):

```
{
  "cpu": {
    "usage": 35.5,
    "cores": 8,
    "temperature": 45.2
  },
  "memory": {
    "total": 16384,
    "used": 8192,
    "available": 8192,
    "percentage": 50.0
  },
  "disk": {
    "total": 512000,
    "used": 256000,
    "available": 256000,
    "percentage": 50.0
  },
  "network": {
    "rx_bytes": 1024000000,
    "tx_bytes": 512000000
  }
}
```

---

## Harmony OS Specific Endpoints

### Distributed Computing

#### Get Cluster Status

**Endpoint:** GET /api/v1/harmony/cluster/status

**Response** (200 OK):

```
{
  "master_node": {
    "id": "harmony-master-1",
    "status": "active",
  }
}
```

```
    "uptime": 86400
  },
  "worker_nodes": [
    {
      "id": "harmony-worker-1",
      "status": "active",
      "tasks_completed": 150,
      "current_load": 0.65
    },
    {
      "id": "harmony-worker-2",
      "status": "active",
      "tasks_completed": 142,
      "current_load": 0.58
    }
  ],
  "total_nodes": 3,
  "active_nodes": 3
}
```

## Distribute Task

**Endpoint:** POST /api/v1/harmony/distribute

**Request:**

```
{
  "task_id": "task-456",
  "distribution_strategy": "round_robin",
  "required_capabilities": ["gpu", "ai_acceleration"]
}
```

**Response (200 OK):**

```
{
  "distributed": true,
  "assigned_nodes": ["harmony-worker-1", "harmony-worker-2"],
  "estimated_completion": "2025-11-07T11:30:00Z"
}
```

## Cross-Device Sync

**Endpoint:** GET /api/v1/harmony/sync/status

**Response (200 OK):**

```
{
  "sync_enabled": true,
  "last_sync": "2025-11-07T10:00:00Z",
  "sync_interval": 30,
  "synced_devices": [
    {
      "device_id": "device-phone-1",
      "device_type": "phone",
      "last_sync": "2025-11-07T10:00:00Z",
      "status": "synced"
    },
    {
      "device_id": "device-tablet-1",
      "device_type": "tablet",
      "last_sync": "2025-11-07T09:59:00Z",
      "status": "synced"
    }
  ]
}
```

## AI Acceleration

**Endpoint:** GET /api/v1/harmony/ai/acceleration

**Response** (200 OK):

```
{
  "npu": {
    "available": true,
    "model": "Kirin 9000",
    "utilization": 45.5
  },
  "gpu": {
    "available": true,
    "model": "Mali-G78",
    "utilization": 62.3
  },
  "optimizations": {
    "quantization": "enabled",
    "pruning": "enabled",
    "precision": "FP16"
  }
}
```

---

# Error Responses

All error responses follow this format:

**Response** (4xx or 5xx):

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid task priority",
    "details": {
      "field": "priority",
      "allowed_values": ["low", "normal", "high", "critical"]
    }
  },
  "request_id": "req-abc123"
}
```

## Common Error Codes

| Code                | HTTP Status | Description                             |
|---------------------|-------------|-----------------------------------------|
| UNAUTHORIZED        | 401         | Missing or invalid authentication token |
| FORBIDDEN           | 403         | Insufficient permissions                |
| NOT_FOUND           | 404         | Resource not found                      |
| VALIDATION_ERROR    | 400         | Invalid request data                    |
| CONFLICT            | 409         | Resource conflict (e.g., duplicate)     |
| RATE_LIMIT_EXCEEDED | 429         | Too many requests                       |
| INTERNAL_ERROR      | 500         | Server error                            |
| SERVICE_UNAVAILABLE | 503         | Service temporarily unavailable         |

# Rate Limiting

API requests are rate limited based on authentication:

- **Authenticated:** 1000 requests/hour
- **Unauthenticated:** 100 requests/hour

Rate limit headers are included in all responses:

X-RateLimit-Limit: 1000  
X-RateLimit-Remaining: 950  
X-RateLimit-Reset: 1699360800

---

## Webhooks

### Register Webhook

**Endpoint:** POST /api/v1/webhooks

**Request:**

```
{  
  "url": "https://example.com/webhook",  
  "events": ["task.completed", "task.failed"],  
  "secret": "your-webhook-secret"  
}
```

**Response (201 Created):**

```
{  
  "id": "webhook-123",  
  "url": "https://example.com/webhook",  
  "events": ["task.completed", "task.failed"],  
  "created_at": "2025-11-07T10:00:00Z"  
}
```

### Webhook Payload Example

```
{  
  "event": "task.completed",  
  "timestamp": "2025-11-07T11:00:00Z",  
  "data": {  
    "task_id": "task-456",  
    "status": "completed",  
    "result": {  
      "success": true,  
      "output": "Task completed successfully"  
    }  
  },  
  "signature": "sha256=abc123..."  
}
```

---

# SDK Examples

## cURL

```
# Login
curl -X POST http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"admin123"}'

# Create task
curl -X POST http://localhost:8080/api/v1/tasks \
  -H "Authorization: Bearer TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"title":"Test Task","type":"planning","priority":"high"}'
```

## Python

```
import requests

# Login
response = requests.post(
    "http://localhost:8080/api/v1/auth/login",
    json={"username": "admin", "password": "admin123"}
)
token = response.json()["token"]

# Create task
headers = {"Authorization": f"Bearer {token}"}
response = requests.post(
    "http://localhost:8080/api/v1/tasks",
    headers=headers,
    json={
        "title": "Test Task",
        "type": "planning",
        "priority": "high"
    }
)
print(response.json())
```

## Go

```
package main
```

```
import (
```

```

"bytes"
"encoding/json"
"net/http"
)

func main() {
    // Login
    loginData := map[string]string{
        "username": "admin",
        "password": "admin123",
    }
    jsonData, _ := json.Marshal(loginData)

    resp, _ := http.Post(
        "http://localhost:8080/api/v1/auth/login",
        "application/json",
        bytes.NewBuffer(jsonData),
    )

    var loginResp map[string]interface{}
    json.NewDecoder(resp.Body).Decode(&loginResp)
    token := loginResp["token"].(string)

    // Create task
    taskData := map[string]string{
        "title": "Test Task",
        "type": "planning",
        "priority": "high",
    }
    jsonData, _ = json.Marshal(taskData)

    req, _ := http.NewRequest(
        "POST",
        "http://localhost:8080/api/v1/tasks",
        bytes.NewBuffer(jsonData),
    )
    req.Header.Set("Authorization", "Bearer "+token)
    req.Header.Set("Content-Type", "application/json")

    client := &http.Client{}
    resp, _ = client.Do(req)
}

```

---

## Best Practices

1. **Always use HTTPS** in production
  2. **Store tokens securely** (never in code/logs)
  3. **Implement exponential backoff** for retries
  4. **Use pagination** for list endpoints
  5. **Verify webhook signatures** for security
  6. **Set appropriate timeouts** for long-running operations
  7. **Monitor rate limits** to avoid throttling
- 

## Support

- Documentation: <https://docs.helixcode.dev>
- GitHub Issues: <https://github.com/helixcode/helixcode/issues>
- Community Forum: <https://forum.helixcode.dev>